

# Bitwise Operators and its Nifty Applications

Jun Hong Wang

## 1 Introduction

Imagine these two scenarios: you are writing math-heavy code for a small, hardware limited computer, like a Raspberry Pi, or it's a very old computer with fewer optimizations. By using modern programming norms, we could write code that works on a modern computer, but these are older, smaller computers with fewer resources to work with. What can we do? How did programmers do this type of math back then?

## 2 Bitwise Operators

A bitwise operator is a normal operator, but instead of working with strings or base 10 numbers, they operate purely on bits and binary representations of strings and numbers. Although there aren't bitwise mathematical operators like addition or subtraction, these operations can be created by using bitwise logical operators (AND, OR, NOT, XOR) and logical shifts (<<, >>, exact representation varies by language). To understand how these operators work, we must convert base 10 numbers into their binary equivalents.

### 2.1 NOT

NOT takes one argument, and flips the bits. All bits set to 1 are set to 0, and all bits set to 0 are set to 1. Notice that this takes one argument, which makes NOT a unary operator. This isn't the same as boolean not, which is used for values like true and false.

In signed integers, where the leftmost bit represents the sign of the integer, a NOT will change the sign of the number. Keep in mind that for these examples, all of the integers are positive, and no bits represent the sign.

NOT 14 = 17

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 17 | 1     | 0     | 0     | 0     | 1     |

### 2.2 AND

AND takes two arguments, and compares each bit using a logical and operator. Only if both bits are 1 will the result be 1.

14 AND 24 = 8

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 24 | 1     | 1     | 0     | 0     | 0     |
| 8  | 0     | 1     | 0     | 0     | 0     |

### 2.3 OR

OR also takes two arguments, and compares each bit using a logical or operator. Only if both bits are 0 will the result be 0.

$$14 \text{ OR } 24 = 30$$

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 24 | 1     | 1     | 0     | 0     | 0     |
| 30 | 1     | 1     | 1     | 1     | 0     |

### 2.4 XOR

XOR takes two arguments, and is like OR, except 1 is only returned when only one of the two arguments has 1 in that bit.

$$14 \text{ XOR } 24 = 22$$

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 24 | 1     | 1     | 0     | 0     | 0     |
| 22 | 1     | 0     | 1     | 1     | 0     |

### 2.5 Left Shift

Left shift moves each bit to the left, and sets the new rightmost bit to a 0. The  $2^3$  bit becomes the  $2^4$  bit, and the new  $2^0$  bit is set to 0. It will take the form  $a \ll b$ , and the result should be  $a * 2^b$ .

$$14 \ll 1 = 28$$

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 28 | 1     | 1     | 1     | 0     | 0     |

### 2.6 Right Shift

Right shift moves each bit to the left, and sets the new leftmost bit to a 0. The  $2^4$  bit becomes the  $2^3$  bit and so on, and the rightmost bit, the  $2^0$  bit, gets cut off.

$$24 \gg 1 = 12$$

|    | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|----|-------|-------|-------|-------|-------|
| 14 | 0     | 1     | 1     | 1     | 0     |
| 22 | 0     | 1     | 1     | 0     | 0     |

### 3 Why They Are Important

Most programmers won't run into these operators unless they go very deep into programming or into certain niches, so why should the average programmer care? After all, the normal operators we've been taught work perfectly fine. However, on some legacy machines or resource limited computers, there is a benefit to using bitwise operators. On old computers, bitwise operators are faster, and also use less resources and power.

### 4 Uses

Uses: A normal implementation of an `isEven` function which takes an integer input and determines whether the input is even and would use `%2` (modulus 2, or remainder after dividing by 2), but one interesting implementation uses logical shifts.

Pseudocode:

```
input n
m = n >> 1
m = m << 1
return m == n
```

This works because if a number is shifted to the right, the rightmost bit is removed, and when the number is shifted back to the left, a 0 gets added. In even numbers, the rightmost bit is always a 0, so when the shifts happen, the end result is the same as the input, so the function will return true.

Another use is addition. The numbers below are in binary, not base 10.

$1 + 1 = 10$ , the  $2^0$  bit of the output is the  $2^0$  bits XOR each other, while the  $2^1$  bit of the output is the  $2^0$  bit AND each other. The XOR part is the sum, while the AND is the carry.

Pseudocode:

```
a and b as input
while b != 0
  carry = a AND b
  a = a XOR b
  b = carry << 1
return a
```

This might look like a bunch of nonsense, but this is roughly what the circuit for addition looks like, which is why historically, this was used instead of base 10 addition. Here, the sum is stored inside `a`, and a left shifted carry is stored in `b`. Ultimately, `b` will become 0, and the values of the other variables will not change after this, so `a`, the sum, is returned.

Here, `0b` specifies that this is the binary representation of a number.

There are also other uses, like cryptography, or bitcoin, where maximum efficiency for calculations is required. Not only that, but by combining some bitwise operators, you can use them for security purposes.

```

>>> bitwiseAdd(6,10)
0b10 carry
0b1100 a
0b100 b
0b100 carry
0b1000 a
0b1000 b
0b1000 carry
0b0 a
0b10000 b
0b0 carry
0b10000 a
0b0 b
16

```

## 5 Bitwise Kenken

In this kenken puzzle, the numbers in each square will be represented in binary. Each row and column must contain numbers from  $1_{10}$  to  $6_{10}$ , or  $001_2$  to  $110_2$  exactly once. Applying the operation on each of the bolded group of squares must result in the number (in base 10) to the left of the operation.

For example, look at *5 NOT* on the leftmost column.  $5_{10} = 101_2$ , and **010** NOT = 101.

For all numbers, assume that they are 3-bit numbers (only 3 binary digits). For bitwise operators, the inputs may be in any order.

|              |                   |                     |                    |                   |              |
|--------------|-------------------|---------------------|--------------------|-------------------|--------------|
| <b>8 ×</b>   |                   |                     | <b>40 &lt;&lt;</b> |                   | <b>1 -</b>   |
| <b>5 NOT</b> | <b>7 XOR</b>      |                     | <b>3 &gt;&gt;</b>  |                   |              |
| <b>3 XOR</b> | <b>6 &lt;&lt;</b> |                     | <b>7 OR</b>        |                   | <b>0 AND</b> |
|              | <b>2 AND</b>      | <b>6 NOT</b>        | <b>6 OR</b>        |                   |              |
| <b>12 +</b>  |                   | <b>192 &lt;&lt;</b> | <b>0 &gt;&gt;</b>  | <b>1 &gt;&gt;</b> |              |
|              |                   |                     |                    | <b>4 &lt;&lt;</b> |              |

## 5.1 Solution

|                          |                               |                                 |                                |                               |                          |
|--------------------------|-------------------------------|---------------------------------|--------------------------------|-------------------------------|--------------------------|
| <b>8 ×</b><br>001<br>1   | 100<br>4                      | 010<br>2                        | <b>40 &lt;&lt;</b><br>101<br>5 | 011<br>3                      | <b>1 -</b><br>110<br>6   |
| <b>5 NOT</b><br>010<br>2 | <b>7 XOR</b><br>011<br>3      | 100<br>4                        | <b>3 &gt;&gt;</b><br>001<br>1  | 110<br>6                      | 101<br>5                 |
| <b>3 XOR</b><br>110<br>6 | <b>6 &lt;&lt;</b><br>001<br>1 | 011<br>3                        | <b>7 OR</b><br>010<br>2        | 101<br>5                      | <b>0 AND</b><br>100<br>4 |
| 101<br>5                 | <b>2 AND</b><br>010<br>2      | <b>6 NOT</b><br>001<br>1        | <b>6 OR</b><br>110<br>6        | 100<br>4                      | 011<br>3                 |
| <b>12 +</b><br>100<br>4  | 110<br>6                      | <b>192 &lt;&lt;</b><br>101<br>5 | <b>0 &gt;&gt;</b><br>011<br>3  | <b>1 &gt;&gt;</b><br>001<br>1 | 010<br>2                 |
| 011<br>3                 | 101<br>5                      | 110<br>6                        | 100<br>4                       | <b>4 &lt;&lt;</b><br>010<br>2 | 001<br>1                 |

## References

”Bitwise Operators in Java.” GeeksforGeeks. April 17, 2023.

<https://www.geeksforgeeks.org/bitwise-operators-in-java/>

Kodichath, Sreedev. “Real World Uses of Bitwise Operators.” Tarka Labs Blog. September 3, 2020.

<https://blog.tarkalabs.com/real-world-uses-of-bitwise-operators-c41429df507f>

Narein, Adith. “Addition using Bitwise Operations”. OpenGenus IQ.

<https://iq.opengenus.org/addition-using-bitwise-operations/>.